Automated Visual Inspection of Packaged Pork Racher Trays

MMA3001 Individual Project

Rory Hallinan-Johns (36253820)

Repo link: https://github.com/rhal0027/MMA3001-Project-repository

1. Engineering Problem

Context

The context of the project was to develop and evaluate an automated method to decide whether packaged meat passes the quality assurance tests. This is for the safety of the meat, since meat that is not properly packed can mean that the shelf life is worse and can create customer complaints that impacts the public view of the business. Even though manual inspection is more simple, these results can be inconsistent because of human error and fatigue [8]. Whereas with automated there is less human error and most meat tray packers normally do 60 portions of meat in a minute [7].

Problem

The problem that had to be solved, is to detect and locate defects in the packaging process. Then from one single camera frame, it had to decide whether the meat would pass or fail the quality assurance test.

Intended use

The use of this project is for end of line decision making aid in order to reject trays for recheck.

Limitations

The limitations of the selected scope is that the camera is only taking single frames of the individual meat trays, with only a single camera set up. There are 5 classes of error in the meat packing including; packaging-error, unsealed tray, twisted meat, loose meat, and wrinkle, other defects are not in the scope. Plus, it is a public dataset so there was no control in the data collection, so there are errors like 3 out of 12 unlabelled test images having missing defects.

2. Inputs and Outputs

Inputs

The input accepted by the solution is one 720x540 RGB 8-bit image of each tray, that is then resized to 640px on the longer side by the model.

Outputs

The system gives two outputs, the first a list of detected defects and a single pass/fail decision based on that list. For each defect found in the image it makes, a bounding box around the defect in the pixels of the 720x540 image. It is in a box format so with centre x and y, width, height in pixels, it is done in order to show an operator where the defect is. The class of the defect, like, packaging error, unsealed, twisted meat, loose meat or wrinkle. This is done so that the right fix is made, reseal or repack. Then it states the confidence, or how sure the model is that the box is a defect, it is a number between 0 to 1 where 1 is certain, used to make the pass-fail decision. The image can have none or several boxes.

Then for the tray decision, the tray is a fail if a detected box has a confidence of 0.12 higher. The 0.12 threshold was chosen because it is the highest confidence that allows at most 2% of defective trays through, to increase the safety of the food.

Invalid inputs

There are also invalid inputs, a black or blurry image, or one with no tray, gives no boxes and is therefore passed. This is a risk because the system can't tell the difference between

images that are black and images that have no defects. So it will pass on them even if the trays do have defects, creating errors in the outputs. Plus many trays were only partly shown, so the model is trained to treat “not much in view” as zero defect.

Data

The data came from a public website called Roboflow, it came from the v4 of the data set entitled, “Pork Rasher Error Packaging”, uploaded by the hello-2aqe0. [3] The data included 3,349 training images from 1085 distinct photos for learning the models weights. 120 validation images for choosing settings during development, like picking the model and the threshold. Finally, there was 80 test photos, for the last independent measure of the performance held back until the end and only used once. For the training data, they were 3x augmented, so the saturation and exposure of the images were changed, so the model is robust to lighting changes in the test data Also the splits were pre-made by Roboflow so were audited before the training was done.

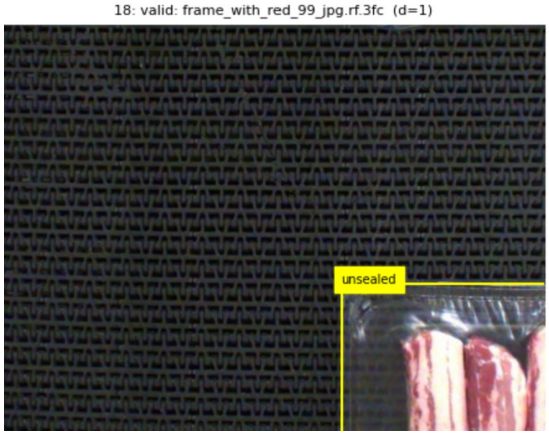


Figure1: Example test image (720x540)

3. Computational approach

YOLO

YOLO is a “You Only Look Once” model which, instead of guessing many regions that may contain an object, runs a single neural network over the entire image and gets its predictions in one go, which is much faster. It extracts visual features, such as edges, textures and shapes, from the 640-pixel image. The image is split into grids at three scales, 80×80, 40×40 and 20×20, with the finer grids catching small defects and the coarser grids finding larger defects. To get one box per defect, YOLO takes the box with the highest confidence, then removes any other box that overlaps it heavily. It then uses COCO to pre-train data so that it is able recognise general features; for example, it can tell when a texture changes or there is an edge. It also means that the output is better, since only 1,085 distinct photos means there is not enough data to train from scratch.

Why YOLOv8n?

1. **Fast:** A production line needs a decision for every tray before the next one arrives. Since YOLO is single-pass and nano is its lightest version, it is very fast.
2. **Mature:** YOLOv8 is widely used and well documented, and the Ultralytics library handles training and export in a few lines. This means less risk of implementation bugs and more time for validation [2].

3. **Trains in under an hour on a free GPU:** Since it is so quick to train on a basic GPU, it is well suited to a student project. It allowed the smoke test, baseline and comparisons to run within Colab's free GPU limits. For example, the baseline stopped at epoch 40 after about 35 minutes on a T4.
4. **Gives locations:** Compared with a simpler alternative that can only classify the whole image as "defective" or "fine", YOLOv8 can give the location of the defect and which defect it is. This allows an operator to fix the problem immediately.

Settings

The images were resized to 640×480 to match the Ultralytics standard size. The number of images processed together before the weights were updated was 16, so that they fit in the T4's memory. The maximum number of epochs (passes through all images) is 50, since this is enough for a pretrained model to converge on a dataset. If the validation score has not improved for 15 epochs in a row, the run stops early in order to avoid over fitting. There is a random seed, so that in every epoch the order changes, so the model does not learn from the order of the images and gets more variety.

Assumptions

It was assumed that all labels are correct except for the three found defects, that the test frames are representative, and that there were not multiple frames of one tray. If incorrect, any of these would affect the data collected and introduce errors, only one of these were able to be tested in the leakage audit.

Implementation

A Python package with six modules was made for the project. A package was used instead of one large notebook because it can be tested, reused by every notebook and documented, which streamlines the process. Three Colab notebooks were then used to run the package, one for each of the three main parts of the project: the data checks, the training and evaluation of YOLOv8n, and the model comparison and optimisation. The notebooks run on Google Colab's free T4 GPU, and training runs are saved to Google Drive so that a disconnect does not lose them. Fifty-three tests were written to check that the code does what it is meant to do, for example that it reads labels correctly. Finally, pdoc documentation was used to explain inputs, outputs and errors; pdoc turns this into web pages for ease of reading.

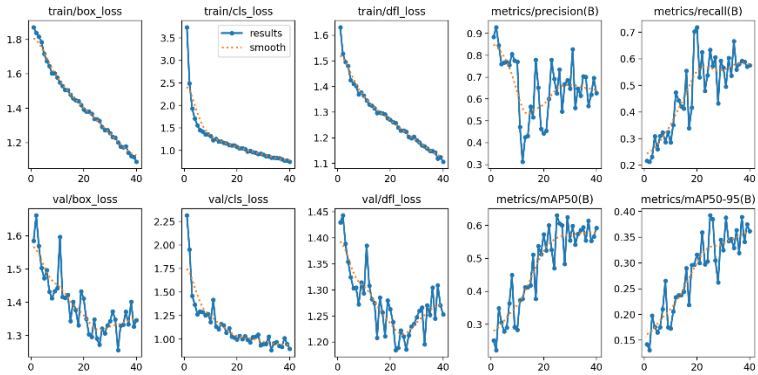


Figure: validation plateaus while training loss false: overfitting caught by early stopping

4. Validation

4a: Data Validation

Leakage check

In order to check that the same tray doesn't appear in both the training and test set, leakage checks had to be completed. If an image was in both the the model has pretty much already seen the answers and results before the test had been done, so its test score looks better then it actually is. Since Roboflow made the splits, they should be checked to ensure that there is no leakage.

Check 1: Filename clashes

Roboflow names each image with a different image, in order to identify the original image. Thus the same name appear in two different splits suggests leakage. Thirteen images with the same name appears in more then one split (4 in train and test, 8 in train and validation and 1 in validation and test). Then all thirteen pairs were manually checked by eye to make sure they weren't the same image, it was found that all thirteen pairs were different images. This shows that filenames aren't a reliable piece of evidence in either direction, meaning the next three checks have to compare the images.

Check 2: Perceptual hash, tested and rejected

This check method shrinks each image down to a 9x8 greyscale image and records 64 bits, comparing whether each pixels is brighter then the neighbour. Similar images give similar bit patterns, the distance between bits can then by compared ro measure how alike images are. This was tested by comparing the augmented copies distanced with the distance from each image to its closest tray. At 64 bits the groups overlapped completely and at a larger size of 256 bits, they were still overlapping. Thus no threshold could separate them, in the first test, it flagged 179 out of 200 images with none of them being duplicates. This failed because all trays are filmed from the same position and look almost identical at low resolution like at 64 bits.

Check 3: Neighbouring video frames

This check makes sure that neighbouring frames of the video aren't accidently of the same tray but slightly moved down. Thus it was measured how close each test or validation number is to the nearest training frame. From this test it was found that the median gap was three frames, this is what a random split by frame makes, so it looked like leakage from these statistics. The 20 closest pairs, all within 4 frames were all checked manually by eye, and it came out that all of these were different pieces of meat. This shows that frame numbers don't track a single tray over time, so either the video was sample sparsely, or multiple recordings share the same numbering, also explaining the filename checks.

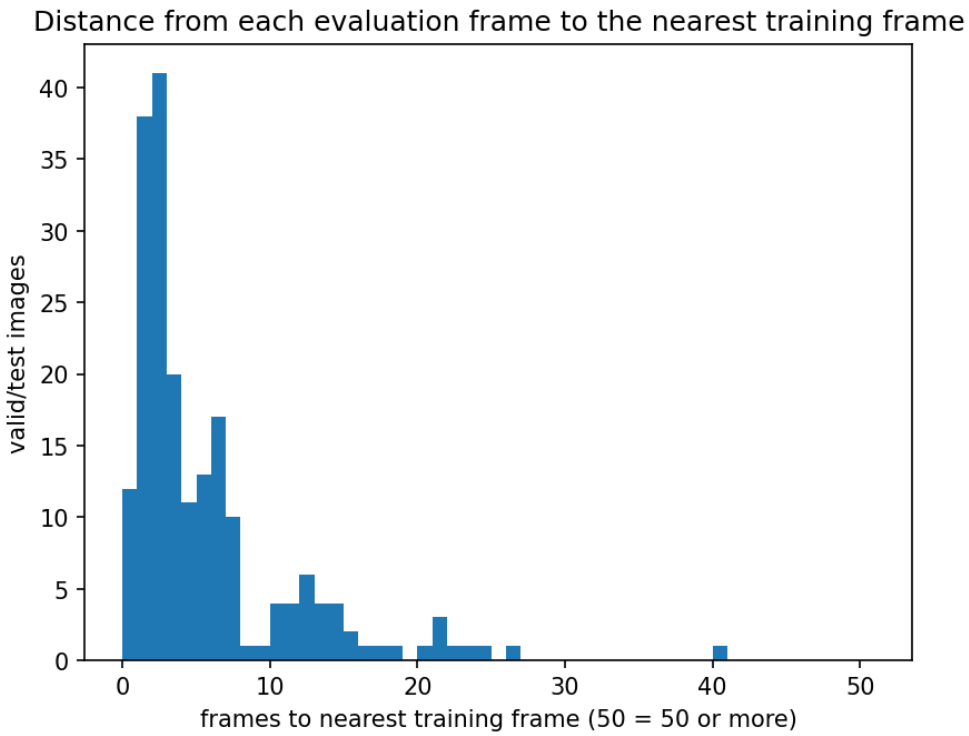


Figure 3: The gap between each testing frame to the nearest training frame, shows quite a small distance on average.

Check 4: Pixel correlation

This compare the images directly, each image gets shrunk to 96 x 72 greyscale and compared with each image using correlation, with 1 meaning identical apart from brightness. The 20 most similar pairs with correlation of 0.91 to 0.93, were all checked by eye and were all different images.

This means after 4 checks there was no evidence of leakage in the data set.

Label quality

Since no label on the images is the main measure for when you pass or fail a test set, a missing label could be detrimental to the, making the model look wrong when its right. In images in training show what a good tray should look life for the test, so if the missing labels are found in the data, then the model learns to ignore defects. For test data all 12 unlabelled images were tested and reviewed manually by eye, and for traing data, a random sample of 24 images was manually tested. From the test data, 3 out of the 12 images had missed defects, with the decided on the confidence interval of 95% the missed-defect rate of the testing images was between 9 and 53%. The training data showed no missed defects, leaving a defect rat of 0-14% in the traing data. The training data looked clean so no re-labelling was needed. The test set had a few errors which were corrected so they would not be passed. This is not definitive though because a sample of 24 cannot rule out small missed defect rate. Plus the reviews was only carried out by one person so it is not definitive.

4b. Model validation

Detection results

mAP50 measures how well a model finds and places moxes, a predicted box counts as correct if a true box is overlapped by over 50%, running from 0-1 with the high the better. mAP50-95 is the same but goes from 50% to 95%. The results showed that the mAP50 value

was 0.5 and the mAP50-95 was 0.27. Unsealed meat was detected the best with 0.87, packaging error was ok with 0.42, twisted meat was never found and loose and twisted meat only have 3-4 test boxes so these scores are not reliable.

Confusion matrix

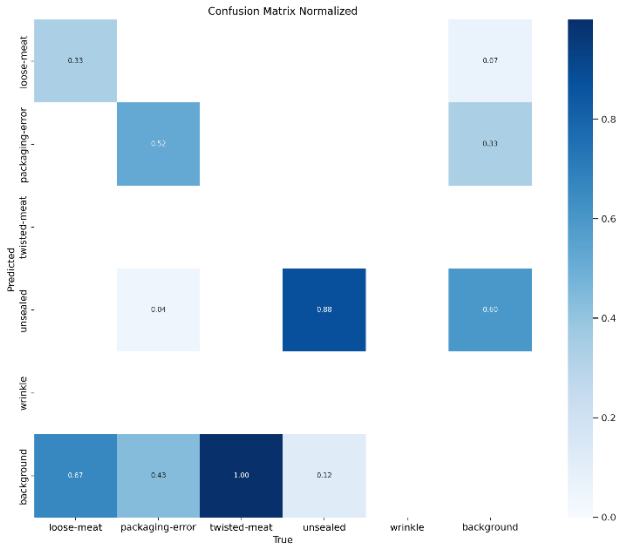


Figure 4: shows the model predicted vs true labels, 43% of packaging errors were missed and predicted as background, unsealed is also predicted but there is no defect

Pass/Fail

0.12 was the chosen value for validation, to allow at most 2% of false passes, then used unchanged on test so the test result is fair. The number of false passes in the validation set is 2/106 or 1.9% (1-7% with confidence interval) and the false fail is 9/14 or 64% (39-84%). Whereas the test with corrected labels has a false pass of 6/71 or 8.5% (4-17%) and false fail of 2/9 or 22%(6-55%).

False passes rose on the test so the threshold doesn't fully carry over to new data. This could have happened because of the cameras, blur, or lighting that may not be controlled for both test.

5. Comparison and Optimisation

Alternatives

The alternatives chosen were YOLOv8s at 640 pixels, this is similar to the baseline, but it is a bit bigger, now it has 11.1 M parameters instead of 3.0 M thus it tests whether more model capacity helps with the more intricate classes. YOLOv8n at 960 pixels was also chosen as another alternative, it is the same small model but the images are enlarged to 960 pixels on the large side. No detail is added, but the defects now cover more pixels which can help with small or thin defects. Each change one thing from the baseline, so the difference can be put onto by that. They also are both supposed to increase the accuracy, since the original model was already quite fast and was not very accurate.

Time budget

A time budget was set since the machine likely does 60 trays a minute [7] so 1000ms per tray, with a 10x safety margin, it only leaves the model 100 ms per image.

The selection criteria before the results were seen was the highest validation mAP-50-95 among models fast enough for the 100ms budget. This was chosen in advance because it stops bias after the results are seen.

	YOLOv8n 640 (baseline)	YOLOv8s 640	YOLOv8n 960
Parameters (M)	3.0	11.1	3.0
GFLOPs per image	8.1	28.4	18.2
Validation mAP50-95	**0.391**	0.317	0.383
Test mAP50-95	**0.273**	0.168	0.267
Validation false fail (at ⩽ 2% false pass)	64%	64%	**29%**
GPU ms per image (T4)	**10.4**	15.1	17.3
CPU ms per image	**99**	249	207

Figure 5: Extract of the comparison table between the 3 options

Meaning of the columns:

Parameters and GFLOPs- model size and computation needed per image, or the memory use and cost

Validation mAP50-95- used to choose the model

Test mAP50-95- Reported for interest, not to choose

Validation false fail- how many good tray the models would reject at the same 2% false-pass limit

GPU and CPU time- speed, compared against the 100ms budget. Only the baseline fits on the CPU.

Figure 4 shows that YOLOv8s is worse on every measure, likely overfitting rare cases. YOLOv8n at 960px rejects a lot fewer good trays but ties on the mAP50-95 and needs twice the computation. Thus, the best model for the project is the baseline (YOLOv8n 640px) since it was the best according to the selection criteria since it had the best mAP50-95 and was the only model to be under the budget of 100ms on CPU.

Optimisation

In order to optimise model, FP16 was used on the GPU, this meant that the numbers were now stored in 16bits not 32bits. The T4 GPU was able to run it, so the model ran 1.63x faster, with no change in accuracy, this was then adopted.

ONNX was then tested on the CPU, it is a standard model that a separate engine can run. It was predicted to speed up the CPU but it was slightly slow and less accurate, likely because It was exported with 640x640 input when it was 640x480, or the new exporter may have introduced changes.

6. Limitations and lessons

The rare cases don't have enough data

Only 72 twisted meat, 66 loose meat, and 18 wrinkle training boxes, with 0 twisted meat recall. Thus the model can't learn these defects properly and can't measure them reliably. T improve as many more examples of these defects should be collected in order to improve learning.

Few clean trays

Only 9-14 trains were clean in validation and test split, with many only partially in view. This means the false fail rate has a large interval and the model may treat the tray not in view properly as clean. To improve need to get photos with meat fully in view.

Selection metric doesn't match the goal

mAP chose the baseline but the 960pixels model was a lot more accurate on rejection of good trays. Thus the model was chosen on box accuracy, but what actually matters for the project is about the pass/fail. In the future the models should be chosen by validation pass/fail instead.

Errors in the test labels

When checking the unlabelled test trays, 3-12 of them came out as missing defects. Test scores are slightly wrong still because the pass/fail was corrected but not the boxes, like twisted meat ect. In the future the trays should be relabelled the test set with a separate reviewer.

7. Ai use reflection

Tools used:

Claude, used through claude.ai, and connected to the github repo.

What it was used for

- Writing python packages, for the tests, the three notebooks, the readMe and docs
- Suggestions on analysis; like leakage test, the label review and threshold rule
- Helped the understanding in concepts like YOLO and ONNX

Approximate level of Ai contribution

Code- mostly Ai Written

Analysis plan- Suggested by Ai but checked and decided by me

Running of code, visual judgements- Both done by me

Report text- written by me with help from readMe and consultation from Ai on how to word complex ideas

Why Ai was used

Ai was used in this case because there wasn't much time to complete the assignment, and learn all the new content like the way of coding and using new tools like YOLO and github. I would rather have focussed on validation and deciding on what route to go down when completing the project. The coding and understanding would have been too hard to do together in the short time from that I gave myself.

How I checked the Ai's output

Looked at and researched all the outputs that were given from the Ai's code. Made visual checks for myself, like the 13 filename pairs. I also questions things that I wasn't sure about, like "are you sure I want python in my gitignore", to ensure I was on the right track.

Decisions that were mine

Choosing meat-packing dataset, visual verdicts, keep 100ms budget when it was suggested to change, the report structure, keeping with the pre-stated selection rule.

Ai's errors and limitations

Duplicate-check flagging 179 out of 200 when none were,

bug recorded 3 missed defects as clean,

The zero false passes rule was impossible

Training time was underestimated
Interrupted training runs would have been skipped if it wasn't checked
It can't look at the images to help tell if it is a missed defect

How Ai affected my understanding and workflow

Ai would go through what it was trying to do before or after every commit which meant I was able to look through each step and decision that it was making so I was able check that this was the correct decision. Even though it wrote and committed most of the coding, it allowed me to critically think about the decisions and learn with the help of it. So, we could work together and try to make the best decisions and project possible. It also allowed me to question what it was doing like skipping training runs that were interrupted.

8. References

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[2] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," version 8.3.40, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[3] hello-2aqe0, "Pork Rasher Error Packaging dataset," version 4, Roboflow Universe. [Online]. Available: https://universe.roboflow.com/hello-2aqe0/pork-rasher-error-packaging (accessed Oct. 22, 2026). Licence: CC BY 4.0.

[4] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.

[5] R. Hallinan-Johns, "MMA3001-Project-repository," GitHub, 2026. [Online]. Available: https://github.com/rhal0027/MMA3001-Project-repository

[6] Anthropic, "Claude," AI assistant, 2026. [Online]. Available: https://claude.ai

[7] Weber Food Technology, "Intelligent complete line solutions from a single source," Weber Food Technology. [Online]. Available: https://www.weberweb.com/our-solutions/concepts/ (accessed Oct. 25, 2026).

[8] G. M. Hancock, "Psychophysical Equivalence of Static Versus Dynamic Stimuli in a Two-Alternative Forced-Choice Detection Task," Academic Press, 2018, pp. 165–168. doi: 10.1016/B978-0-12-811926-6.00027-0.